



**FACULTAD DE
INGENIERÍA**
UNIVERSIDAD DA VINCI
DE GUATEMALA

Universidad Da Vinci De Guatemala

Facultad de Ingeniería

Carrera: Ingeniería en Sistemas



**FACULTAD DE
INGENIERÍA**

UNIVERSIDAD DA VINCI
DE GUATEMALA

“Evaluación Final – Fase 1”

Adela Margarita González Chamalé

Carné: 202500214

Curso: Estructura de Datos

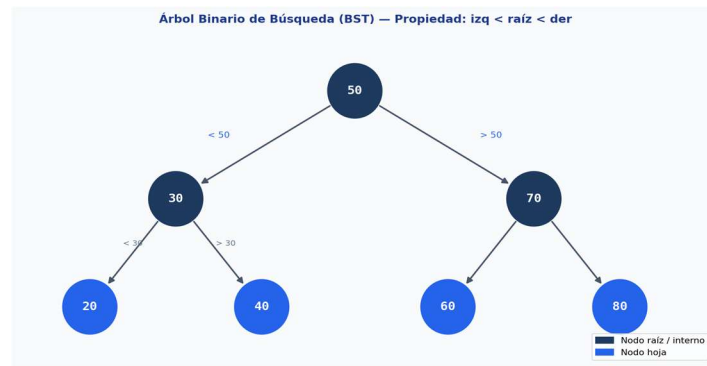
Guatemala, mayo de 2026

Árbol Binario de Búsqueda (BST)

Identificación

Árbol donde cada nodo tiene como máximo dos hijos. El subárbol izquierdo contiene valores menores al nodo raíz y el subárbol derecho contiene valores mayores. Esta propiedad se cumple recursivamente en cada nodo del árbol.

Diagrama



Análisis de Complejidad (Big O)

Operación	Caso Promedio	Peor Caso
Búsqueda	$O(\log n)$	$O(n)$
Inserción	$O(\log n)$	$O(n)$
Eliminación	$O(\log n)$	$O(n)$

Características Mecánicas

Cada inserción compara el valor nuevo con el nodo actual, si es menor, se mueve al hijo izquierdo; si es mayor, al derecho. Este proceso se repite hasta encontrar un espacio nulo. La búsqueda sigue la misma lógica, reduciendo el espacio de búsqueda a la mitad en cada paso (en árboles balanceados). La eliminación contempla tres casos: nodo hoja (se elimina directamente), nodo con un hijo (se reemplaza por su hijo), y nodo con dos hijos (se sustituye por su sucesor in-order).

Ventajas y Limitaciones

Mayor Ventaja

Permite búsqueda, inserción y eliminación eficiente en $O(\log n)$ para árboles balanceados.

Mayor Limitación

Se degenera a $O(n)$ si los datos se insertan en orden ascendente o descendente, convirtiéndose en una lista enlazada.

Caso de Uso en la Industria

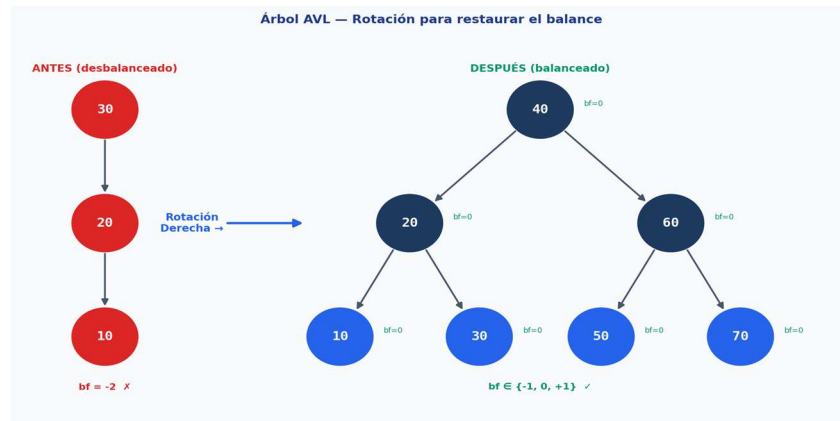
Los sistemas de autocompletado en editores de código (como VS Code) utilizan BSTs para indexar símbolos del proyecto. También se usan en implementaciones básicas de diccionarios, como en intérpretes de lenguajes de scripting para búsqueda de variables en tabla de símbolos.

Árbol AVL (Árbol Balanceado)

Identificación

Variante auto-balanceada del BST donde la diferencia de altura entre los subárboles izquierdo y derecho de cualquier nodo (factor de balance) es siempre -1, 0 o +1. Si tras una inserción o eliminación el factor de balance viola esta regla, el árbol se reequilibra automáticamente mediante rotaciones.

Diagrama



Análisis de Complejidad (Big O)

Operación	Caso Promedio	Peor Caso
Búsqueda	$O(\log n)$	$O(\log n)$
Inserción	$O(\log n)$	$O(\log n)$
Eliminación	$O(\log n)$	$O(\log n)$

Características Mecánicas

Cada nodo almacena su factor de balance (altura derecha - altura izquierda). Tras cada inserción o eliminación se actualizan los factores desde el nodo afectado hasta la raíz. Si algún nodo queda con factor ± 2 , se aplica una rotación simple (izquierda o derecha) o doble (izquierda-derecha o derecha-izquierda) para restaurar el equilibrio. Las rotaciones son operaciones $O(1)$ que reorganizan punteros localmente.

Ventajas y Limitaciones

Mayor Ventaja

Garantiza $O(\log n)$ en todas las operaciones incluso en el peor caso, independientemente del orden de inserción.

Mayor Limitación

Mayor sobrecarga de memoria (almacena altura/factor de balance por nodo) y mayor complejidad de implementación frente a un BST simple.

Caso de Uso en la Industria

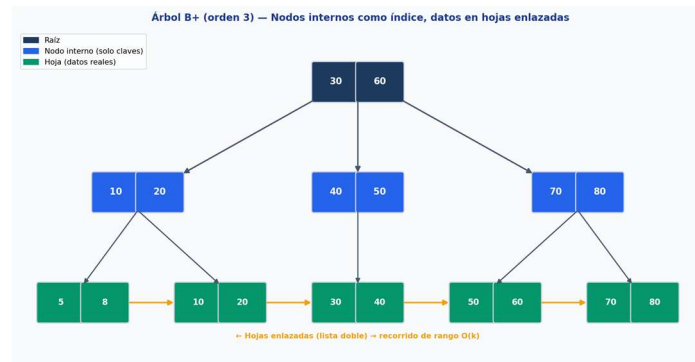
Los sistemas de gestión de memoria de algunas implementaciones del kernel de Linux usan árboles AVL (o rojo-negro) para administrar regiones de memoria virtual. Bases de datos in-memory como Redis utilizan estructuras similares para sus sorted sets, garantizando rangos de consulta $O(\log n)$.

Árbol B / B+

Identificación

Árbol de búsqueda auto-balanceado de orden m donde cada nodo puede contener entre $\lceil m/2 \rceil - 1$ y $m-1$ claves y hasta m hijos. El Árbol B+ extiende el B almacenando todos los datos solo en las hojas, las cuales están enlazadas entre sí formando una lista doblemente ligada para facilitar recorridos secuenciales.

Diagrama



Análisis de Complejidad (Big O)

Operación	Caso Promedio	Peor Caso
Búsqueda	$O(\log n)$	$O(\log n)$
Inserción	$O(\log n)$	$O(\log n)$
Eliminación	$O(\log n)$	$O(\log n)$

Características Mecánicas

El árbol crece desde la raíz hacia arriba. Cuando un nodo se llena ($m-1$ claves), se divide en dos: la clave mediana asciende al padre. Si el padre también se llena, el proceso se propaga hasta la raíz; si la raíz se divide, el árbol gana un nivel. El alto factor de ramificación (miles de hijos por nodo) minimiza la altura total del árbol y, crucialmente, el número de accesos a disco, ya que un nodo equivale a una página de disco.

Ventajas y Limitaciones

Mayor Ventaja

Extremadamente eficiente para almacenamiento en disco. Su alto factor de ramificación reduce la altura a 2-4 niveles para millones de registros, minimizando I/O.

Mayor Limitación

Complejo de implementar correctamente, especialmente el manejo de divisiones y fusiones de nodos. Presenta mayor consumo de espacio por la redundancia de claves en nodos internos.

Caso de Uso en la Industria

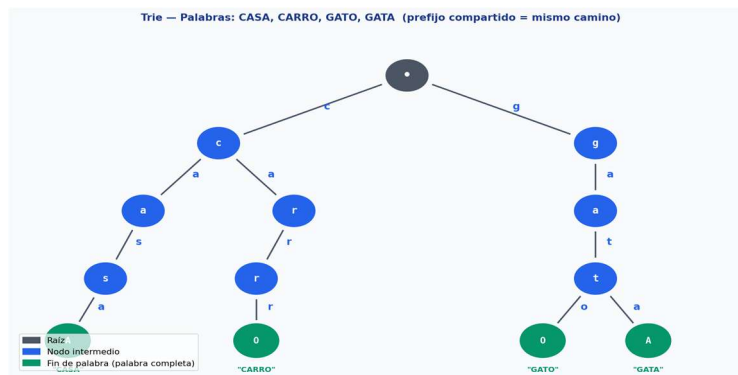
Es la estructura central de los motores de bases de datos relacionales. PostgreSQL, MySQL (InnoDB), Oracle y SQL Server usan árboles B+ para sus índices. Un índice sobre la columna email en una tabla de 10 millones de filas permite encontrar un registro en 3-4 lecturas de disco gracias a esta estructura.

Trie (Árbol de Prefijos)

Identificación

Árbol donde cada nodo representa un carácter de una cadena. La posición de un nodo en el árbol define la clave a la que pertenece: la raíz es la cadena vacía y cada camino desde la raíz hasta un nodo marcado como fin de palabra representa una clave completa. No almacena la clave en el nodo; la clave está implícita en el camino recorrido.

Diagrama



Análisis de Complejidad (Big O)

Operación	Caso Promedio	Peor Caso
Búsqueda	$O(m)$ — m = largo de la clave	$O(m)$
Inserción	$O(m)$	$O(m)$
Eliminación	$O(m)$	$O(m)$

Características Mecánicas

Cada nodo tiene hasta $|\Sigma|$ hijos, donde Σ es el alfabeto (26 para letras, 128 para ASCII). Para insertar 'gato': desde la raíz, se crea/navega el nodo 'g', luego 'a', 't', 'o', marcando el último como fin de palabra. La búsqueda sigue el mismo camino; si algún carácter no existe como hijo, la clave no está en el Trie. Las operaciones son proporcionales a la longitud de la clave, completamente independientes del número de claves almacenadas.

Ventajas y Limitaciones

Mayor Ventaja

La búsqueda de prefijos es $O(m)$ independientemente del número de claves almacenadas.

Mayor Limitación

Alto consumo de memoria: cada nodo puede requerir un arreglo de punteros de tamaño $|\Sigma|$.

Caso de Uso en la Industria

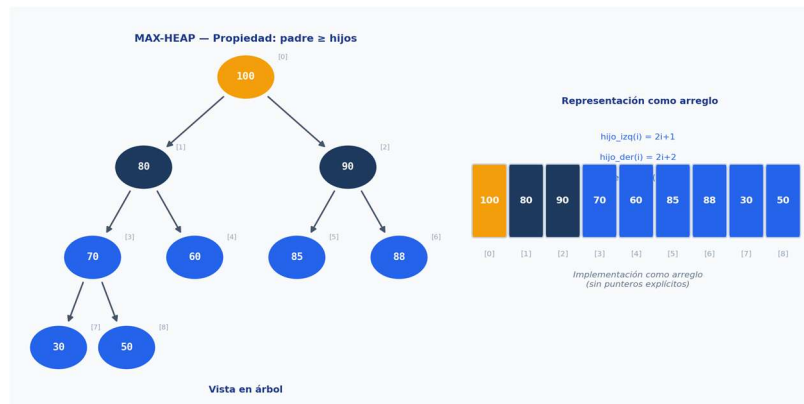
Los motores de búsqueda como Google y los sistemas de autocompletado de teclados móviles (SwiftKey, Gboard) utilizan Tries o estructuras derivadas (Radix Trees) para ofrecer sugerencias en tiempo real. Los routers de red también dependen de esta estructura para longest-prefix matching con direcciones IP.

Heap (Montículo Min/Max)

Identificación

Árbol binario completo que satisface la propiedad de heap: en un Max-Heap, el valor de cada nodo es mayor o igual al de sus hijos; en un Min-Heap, menor o igual. El elemento de mayor (o menor) prioridad siempre se encuentra en la raíz. Se implementa eficientemente sobre un arreglo sin necesidad de punteros explícitos.

Diagrama



Análisis de Complejidad (Big O)

Operación	Caso Promedio	Peor Caso
Peek (raíz)	$O(1)$	$O(1)$
Inserción (push)	$O(\log n)$	$O(\log n)$
Eliminación raíz (pop)	$O(\log n)$	$O(\log n)$
Heapify (construir)	$O(n)$	$O(n)$

Características Mecánicas

Se representa como un arreglo donde el nodo en el índice i tiene sus hijos en $2i+1$ y $2i+2$, y su padre en $\lfloor (i-1)/2 \rfloor$. Al insertar, el nuevo elemento se coloca al final y flota hacia arriba (sift-up) intercambiándose con su padre mientras viole la propiedad de heap. Al eliminar la raíz, el último elemento la reemplaza y hunde hacia abajo (sift-down) intercambiándose con el hijo de mayor prioridad.

Ventajas y Limitaciones

Mayor Ventaja

Garantiza acceso al elemento de mayor prioridad en $O(1)$. Es la implementación más eficiente de una cola de prioridad.

Mayor Limitación

No permite búsqueda eficiente de elementos arbitrarios: encontrar un elemento que no sea la raíz requiere $O(n)$.

Caso de Uso en la Industria

Los planificadores de tareas (schedulers) de sistemas operativos como Linux (Completely Fair Scheduler) usan un heap para gestionar qué proceso ejecutar a continuación.